

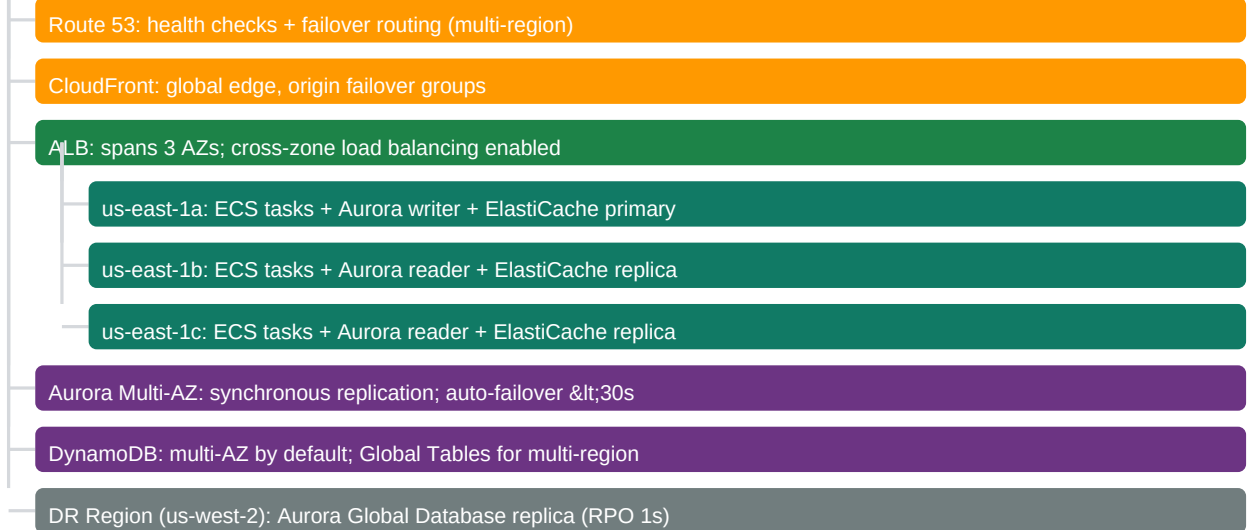
20-00

Chapter Overview — HA and DR at ShopFlow

High Availability (HA) means the system remains operational during failures within a region. Disaster Recovery (DR) means the system can be restored after a catastrophic event (region failure, data corruption, ransomware). ShopFlow targets: 99.99% availability (52 minutes downtime/year), RTO (Recovery Time Objective) of 15 minutes, RPO (Recovery Point Objective) of 1 minute for critical data. This chapter covers: multi-AZ architecture, multi-region failover, Aurora Global Database, backup and restore, and chaos engineering.

Availability Target	Downtime/Year	Downtime/Month	Architecture Required	ShopFlow Status
99.9% (three nines)	8.7 hours	43 minutes	Single-AZ with auto-restart	Below target (not acceptable for e-commerce)
99.95%	4.4 hours	21 minutes	Multi-AZ, no multi-region	Minimum acceptable for ShopFlow
99.99% (four nines)	52 minutes	4.4 minutes	Multi-AZ + automated failover + health checks	ShopFlow target
99.999% (five nines)	5 minutes	26 seconds	Multi-region active-active + global load balancing	Black Friday target for order placement

ShopFlow HA Architecture — Three Availability Zones



20-01

RTO and RPO — Defining ShopFlow Recovery Objectives

RTO (Recovery Time Objective): maximum acceptable time from failure to restoration. RPO (Recovery Point Objective): maximum acceptable data loss measured in time. ShopFlow tier-1 services (order placement, payment) have RTO 15 min / RPO 1 min. Tier-2 (product catalog, search) have RTO 30 min / RPO 5 min. Non-critical (analytics, reporting) have RTO 4 hours / RPO 1 hour.

Service Tier	Services	RTO Target	RPO Target	DR Strategy	Cost Impact
Tier 1 — Critical	Order placement, payment processing	15 minutes	1 minute	Multi-region active-passive; Aurora Global Database	High (\$4,500/mo DB Extra)
Tier 2 — Important	Product catalog, search, user auth	30 minutes	5 minutes	Multi-AZ in primary region; Aurora snapshot restore	Medium (\$1,200/mo for DR Region)

Service Tier	Services	RTO Target	RPO Target	DR Strategy	Cost Impact
Tier 3 — Standard	Analytics, reporting, recommendations	4 hours	1 hour	Daily Aurora snapshot; Kinesis S3 replay from data	Low (\$800/mo for cross-region)
Tier 4 — Non-Critical	Dev tools, admin dashboard, batch jobs	24 hours	24 hours	Single-AZ; no DR; recreate from CDK	None (no DR infrastructure)

20-02

Aurora Global Database — Sub-Second RPO

Aurora Global Database replicates changes from the primary region (us-east-1) to up to 5 secondary regions with ~1 second replication lag (RPO ~1 second). In a region failure, secondary region is promoted to primary in less than 1 minute (RTO). ShopFlow uses Aurora Global Database for order and payment data (Tier 1).

Aurora Global Database CDK

```
// Primary cluster (us-east-1)
const primaryCluster = new rds.DatabaseCluster(this, "ShopFlowPrimary", {
  engine: rds.DatabaseClusterEngine.auroraPostgres({
    version: rds.AuroraPostgresEngineVersion.VER_15_4,
  }),
  instanceProps: {instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G,
    ec2.InstanceSize.XLARGE2)},
  instances: 3, // 1 writer + 2 readers
  storageEncrypted: true,
  deletionProtection: true,
});

// Global Database: add us-west-2 secondary
const globalDb = new rds.CfnGlobalCluster(this, "ShopFlowGlobal", {
  globalClusterIdentifier: "shopflow-global",
  sourceDbClusterIdentifier: primaryCluster.clusterArn,
  deletionProtection: true,
});

// In us-west-2 stack: create secondary cluster attached to global cluster
// Secondary is read-only; promoted to writer only during failover
```

Aurora Global DB Feature	Value	ShopFlow Usage
Replication lag (primary to secondary)	Typically <1 second (tested under load)	RPO: at most 1 second of data loss in region failover
Failover RTO (secondary promoted to primary)	<1 minute (managed failover) or <60 seconds (manual)	RTO target is 15 minutes; Aurora failover is the fastest step
Secondary region read replicas	Up to 15 read replicas in secondary region	ShopFlow uses secondary for read traffic during non-failover
Cross-region data transfer cost	\$0.20/GB replicated (Aurora Global replication)	ShopFlow: ~50 GB/day write volume = \$10/day = \$300/month

20-03

Multi-AZ ECS and Auto Scaling

ShopFlow ECS services are deployed across 3 AZs with a minimum of 2 tasks per AZ (6 total). ALB distributes traffic across all healthy tasks regardless of AZ (cross-zone load balancing). If us-east-1a loses all ECS tasks (AZ failure), ALB detects unhealthy targets within 10 seconds (2 health check failures x 5 seconds) and routes all traffic to tasks in us-east-1b and us-east-1c.

ECS Multi-AZ Placement Strategy CDK

```
const orderService = new ecs.FargateService(this, "OrderService", {
  cluster,
  taskDefinition: orderTaskDef,
  desiredCount: 6, // minimum 2 tasks per AZ
  assignPublicIp: false,
  vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
  spreadTasksByAz: true, // ECS spread tasks evenly across 3 AZs
  capacityProviderStrategies: [{
    capacityProvider: "FARGATE",
    weight: 1,
```

```
base: 6, // always run at least 6 tasks
}},
circuitBreaker: {
rollback: true, // rollback to last known good task def on deployment failure
},
deploymentController: {type: ecs.DeploymentControllerType.ECS},
});
// Auto Scaling: scale out before AZ failure causes overload
const scaling = orderService.autoScaleTaskCount({minCapacity: 6, maxCapacity: 30});
scaling.scaleOnCpuUtilization("CpuScaling",{
targetUtilizationPercent: 60, // scale at 60% CPU (leaves headroom for AZ loss)
scaleInCooldown: Duration.seconds(300), // slow scale-in
scaleOutCooldown: Duration.seconds(60), // fast scale-out
});
```

20-04

Aurora Automated Failover and Read Replica Strategy

Aurora Multi-AZ: writer in us-east-1a, readers in us-east-1b and us-east-1c. Aurora automatically promotes the highest-priority reader to writer if the writer fails. Failover takes 30-120 seconds (DNS endpoint update propagation). ShopFlow order-svc uses Aurora cluster endpoint (writer) for writes and reader endpoint (load-balanced across readers) for reads.

Aurora Endpoint Type	Behavior During Writer Failover	ShopFlow Usage	Connection Pool Setting
Cluster endpoint (writer)	Updates DNS to point to new writer after ~30s failover	order-svc: all INSERT, UPDATE, DELETE operations	Pool size: 10-20
Reader endpoint (load-balanced)	Continues serving reads from surviving readers	product-svc: all SELECT queries; reports writer SREs	Pool size: 5-10 per reader; read-only
Instance endpoint (specific node)	Does NOT auto-failover (pinned to specific instance)	Not used by ShopFlow (use cluster endpoint instead)	Avoided as per AWS best practice of Aurora Multi-AZ
Custom endpoint (subset of readers)	Survives as long as subset reader is healthy	Analytics queries routed to dedicated pool of large readers (not shared)	Pool size: 2-4 large readers

Aurora Connection Retry — Spring Boot

```
spring:
datasource:
url: jdbc:postgresql://${AURORA_CLUSTER_ENDPOINT}:5432/shopflow
hikari:
connection-timeout: 3000 # 3s: fail fast on connection attempt
maximum-pool-size: 20
connection-test-query: SELECT 1
# Aurora failover: ~30s DNS update; retry for 60s total
initialization-fail-timeout: -1 # do not fail on startup if DB unavailable
```

20-05

DynamoDB Global Tables — Multi-Region Active-Active

DynamoDB Global Tables replicate data across multiple regions automatically. Unlike Aurora Global Database (active-passive), DynamoDB Global Tables is active-active — writes can go to any region and are replicated to all others. ShopFlow uses Global Tables for: shopping cart (user expects cart accessible from any region) and user sessions. Last-writer-wins conflict resolution (based on timestamp).

DynamoDB Global Tables CDK

```
const cartTable = new dynamodb.Table(this,"ShoppingCart",{
tableName: "shopflow-cart",
billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
partitionKey: {name: "userId", type: dynamodb.AttributeType.STRING},
sortKey: {name: "productId", type: dynamodb.AttributeType.STRING},
replicationRegions: ["us-west-2", "eu-west-1"], // Global Tables: 3 regions
timeToLiveAttribute: "expiresAt", // cart TTL: 7 days
stream: dynamodb.StreamViewType.NEW_AND_OLD_IMAGES, // required for Global Tables
pointInTimeRecovery: true, // PITR: restore to any second in last 35 days
```

```
});
```

DynamoDB Global Tables vs Aurora Global Database	Aurora Global Database	Aurora Global Database
Replication model	Active-active (write to any region)	Active-passive (write to primary only; secondary is read-only)
Conflict resolution	Last-writer-wins (timestamp-based; application must handle single writes)	No conflict resolution (single writer); secondary is replica
Failover type	No failover needed (already active in secondary region)	Managed failover: promote secondary to primary (<1 min)
Read in secondary region	Read from local DynamoDB (no cross-region latency)	Read from local Aurora read replica (no cross-region latency)
Write in secondary region	Writes replicated to primary and all regions (write to all)	Writes go to primary only (cross-region write latency from secondary)
ShopFlow usage	Shopping cart, user sessions (active-active; use in data, primary read data only)	Order data, payment data (single source of truth; no active)

20-06

Route 53 Health Checks and Failover

Route 53 health checks monitor primary region (us-east-1) ALB every 10 seconds. If health check fails 3 consecutive times (30 seconds), Route 53 automatically updates DNS to point shopflow.com to DR region (us-west-2) ALB. Combined with Aurora Global Database promotion, full failover completes in under 2 minutes for DNS TTL propagation.

Route 53 Failover Routing CDK

```
// Health check: monitors primary ALB every 10 seconds
const primaryHealthCheck = new route53.CfnHealthCheck(this, "PrimaryHealthCheck", {
  healthCheckConfig: {
    type: "HTTPS",
    fullyQualifiedDomainName: primaryAlb.loadBalancerDnsName,
    port: 443,
    resourcePath: "/health", // ALB target group health endpoint
    requestInterval: 10, // check every 10 seconds
    failureThreshold: 3, // 3 failures = unhealthy
    measureLatency: true,
  },
});

// Failover record set: PRIMARY (us-east-1) and SECONDARY (us-west-2)
new route53.ARecord(this, "PrimaryRecord", {
  zone,
  recordName: "shopflow.com",
  target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(primaryAlb)),
  // @ts-ignore CDK does not expose failover routing directly; use CfnRecordSet:
});
```

■ Route 53 DNS TTL matters critically for failover. ShopFlow shopflow.com TTL: 60 seconds. During failover, old DNS records cached in user browsers/CDN for up to 60 seconds. Set TTL to 60 seconds always (not 300 or 3600) — lower TTL means faster failover propagation. Route 53 health check failures propagate to DNS updates in ~60 seconds (3 failed checks x 10s = 30s, plus Route 53 propagation). Total failover: health check (30s) + DNS propagation (60s) + Aurora promotion (60s) = ~2.5 minutes.

20-07

Backup Strategy — Aurora, DynamoDB, S3

ShopFlow backup strategy per data tier: Aurora (automated daily snapshots, 35-day PITR, cross-region copy to us-west-2), DynamoDB (PITR enabled, 35-day continuous backup, on-demand backup before major releases), S3 data lake (versioning enabled, MFA delete for production bucket, cross-region replication for Bronze layer). Backup test: monthly restore drill.

	Backup Method	Retention	RPO	RTO (restore)	Cross-Region
PostgreSQL	Automated snapshots (daily) + PITR (continuous)	35 days	1 second (PITR)	~1 hour per 100 GB table restore	Yes: daily cross-region snapshot
DynamoDB Global Tables	PITR (continuous, 35-day window) + on-demand backup before releases	35 days	1 second (PITR)	~1 hour per 100 GB table restore	Yes: Global Tables replication

	Backup Method	Retention	RPO	RTO (restore)	Cross-Region
ake	Versioning enabled + lifecycle (delete old versions after 90 days)	35 days (effective)	0 (point-in-time)	15 min (access older version via versioning)	Yes: IS3 Cross-Region
ne Redis	Automatic daily snapshots (Redis persisted)	1 day (RDS cache)	24 hours (acceptable)	15 min (restore from snapshot)	No: cache is reconstructed

20-08

AWS Backup — Centralized Backup Policy

AWS Backup provides centralized backup management across Aurora, DynamoDB, EFS, EC2, and RDS with a single policy. ShopFlow uses AWS Backup Vault with a backup plan: daily at 2am, 35-day retention, cross-region copy to us-west-2. AWS Backup integrates with AWS Organizations — apply backup policy to all production accounts via Service Control Policy.

AWS Backup CDK

```
const backupVault = new backup.BackupVault(this, "ShopFlowVault", {
  backupVaultName: "shopflow-production",
  encryptionKey: backupKey, // customer-managed KMS
  accessPolicy: vaultPolicy, // deny delete without MFA
  lockConfiguration: {
    minRetentionDays: 35, // vault lock: cannot delete backups before 35 days
    maxRetentionDays: 2555, // 7 years for financial data
    changeableForDays: 3, // 3-day governance mode; switch to compliance after
  },
});

const backupPlan = new backup.BackupPlan(this, "ShopFlowBackupPlan", {
  backupPlanRules: [
    new backup.BackupPlanRule({
      ruleName: "daily-backup",
      scheduleExpression: events.Schedule.cron({hour:"2",minute:"0"}),
      deleteAfter: Duration.days(35),
      copyActions: [{
        destinationBackupVault: drVault, // us-west-2 vault
        deleteAfter: Duration.days(35),
      }],
    }),
  ],
});

backupPlan.addSelection("AllResources", {
  resources: [backup.BackupResource.fromTag("Environment", "prod")],
});
```

20-09

Chaos Engineering — Testing HA at ShopFlow

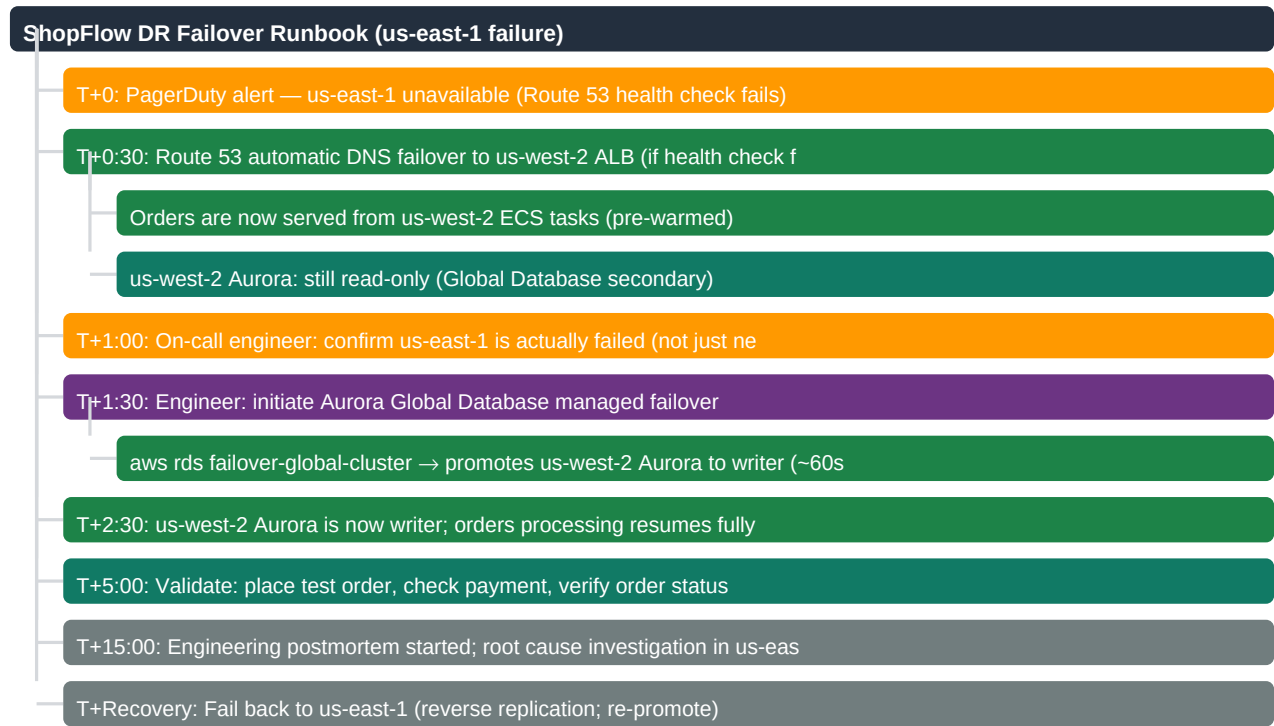
Chaos Engineering deliberately injects failures to validate HA architecture before they happen in production. ShopFlow uses AWS Fault Injection Simulator (FIS) for structured chaos experiments. Monthly GameDay: inject one failure, measure actual RTO/RPO, compare to targets, fix gaps. Common ShopFlow chaos experiments: AZ failure simulation, Aurora failover, ECS task termination, network latency injection.

Chaos Experiment	FIS Action	Expected Behavior	ShopFlow Actual Result	Fix Applied
Terminate 50% of ECS tasks (AZ failure simulation)	aws:ecs:terminate (50% of order-svc tasks)	Auto Scaling replaces tasks; ALB gracefully handles replacement	Actually, 30s code deploy before replacement	Increased health check timeout
Aurora writer failover	aws:rds:failover-db-cluster	App reconnects via JDBC retry; <45 seconds p99	45 seconds p99 DB connection errors (retry loop)	Added retry logic
Network latency injection (300ms)	aws:network:add-network-latency	App degrades gracefully; Bulkhead protects database	Timeout limits impact order-svc times out	Retries do not cascade
Delete all ElastiCache nodes	aws:elasticache:delete-replication-group	App stalls; back to Aurora reads; high DB load	DB load spikes 10x Aurora connections	Added DB capacity

20-10

Multi-Region DR Runbook

ShopFlow DR runbook: documented step-by-step procedure for failing over from us-east-1 to us-west-2. Runbook is executed automatically for first 15 minutes (Route 53 + Aurora promotion), then manually validated by on-call engineer. Runbook is tested monthly in staging; production DR drill quarterly.



20-11

Health Check Architecture — ALB, ECS, and Route 53

ShopFlow health check hierarchy: (1) ECS container health check (process-level; restart container if fails), (2) ALB target group health check (service-level; remove unhealthy task from load balancing), (3) Route 53 health check (endpoint-level; fail over to DR region if ALB unhealthy). Each layer has different granularity, interval, and action.

Check Layer	Checks	Interval	Failure Action	Recovery Action
Container Health	CMD: curl -f http://localhost:8080/actuator/health	30 seconds; 3 retries	Restart container (not terminate task); unhealthy in ECS	Only in ECS
Target Group	HTTP GET /health; expect 200 OK; 5-second timeout	10 seconds; 2 failures = unhealthy	Remove task from ALB target group; traffic redirected to other tasks	2 consecutive failures = unhealthy
Health Check	HTTPS GET to ALB DNS; port 443; /health endpoint	10 seconds; 3 failures = trigger DNS failover	Trigger DNS failover; promote A record to DR region	2 consecutive failures = trigger failover

ECS Container Health Check CDK

```
container.addHealthCheck({
  command: ["CMD-SHELL", "curl -f http://localhost:8080/actuator/health || exit 1"],
  interval: Duration.seconds(30),
  timeout: Duration.seconds(5),
  retries: 3, // 3 consecutive failures = UNHEALTHY
  startPeriod: Duration.seconds(60), // grace period: ignore failures during startup
});
// ALB Target Group Health Check:
const targetGroup = new elbv2.ApplicationTargetGroup(this, "OrderTg", {
  vpc,
  port: 8080,
  protocol: elbv2.ApplicationProtocol.HTTP,
  healthCheck: {
    path: "/actuator/health",
    healthyHttpCodes: "200",
    interval: Duration.seconds(10),
```

```
timeout: Duration.seconds(5),
healthyThresholdCount: 2,
unhealthyThresholdCount: 2, // 2 failures (20 seconds) = unhealthy
},
});
```

20-12

AWS Resilience Hub — Automated Resiliency Assessment

AWS Resilience Hub assesses application architecture against RTO/RPO targets. Import ShopFlow from CloudFormation stacks; set resiliency policy (RTO 15min / RPO 1min); Resilience Hub identifies gaps and recommends fixes. ShopFlow initial assessment: 7 gaps found. Primary gaps: ElastiCache no Multi-AZ, Lambda no DLQ on 3 functions, S3 versioning missing on 2 buckets.

Resilience Hub Resiliency Policy CDK

```
new resiliencehub.CfnResiliencyPolicy(this,"ShopFlowPolicy",{
  policyName: "shopflow-production-policy",
  tier: "MissionCritical",
  policy: {
    "AZ": {
      rtoInSecs: 300, // 5 minutes RTO for AZ failure
      rpoInSecs: 60, // 1 minute RPO for AZ failure
    },
    "Hardware": {
      rtoInSecs: 900, // 15 minutes RTO for hardware failure
      rpoInSecs: 60,
    },
    "Software": {
      rtoInSecs: 300,
      rpoInSecs: 60,
    },
    "Region": {
      rtoInSecs: 3600, // 1 hour RTO for full region failure
      rpoInSecs: 300, // 5 minute RPO for region failure
    },
  },
});
```

■ Resilience Hub scores your application from 0-100. ShopFlow initial score: 62 (gaps in ElastiCache single-AZ, missing DLQs). After fixes: 91. Resilience Hub integrates with CodePipeline — add a Resilience Hub assessment as a pipeline gate: deployment blocked if score drops below 85. This prevents engineers from accidentally removing HA features (e.g., switching Aurora to single-AZ) in the name of cost savings.

20-13

Active-Active vs Active-Passive — ShopFlow Strategy

ShopFlow uses different multi-region strategies per service tier: Active-Active (DynamoDB Global Tables for cart and sessions — both regions serve reads and writes), Active-Passive (Aurora Global Database for orders — one region writes; secondary is standby), Backup-Only (analytics S3 data lake — DR region has copy but no active compute). Strategy selection depends on write conflict tolerance and cost.

Pattern	Both Regions Handle Writes	Complexity	Data Consistency	ShopFlow Use Case	Monthly Cost (DR)
Active-Active	Yes (write to nearest)	High (conflict resolution, eventual consistency)	Eventual consistency (last-write-wins)	DynamoDB cart, ElastiCache sessions (\$800 (DynamoDB Global Tables))	\$800 (DynamoDB Global Tables)
Active-Passive	No (primary writes; secondary standby)	Medium (failover, standby DNS change)	Strongly consistent (no conflicts)	Aurora Global Database; SQS order queue (\$2,100 (Aurora Global Database))	\$2,100 (Aurora Global Database)
Pilot Light	No (secondary has minimal resources)	Low (simple pre-provisioned resources)	Eventually consistent (backup/recovery)	Analytics S3 data lake; reporting (\$400 (DR Aurora snapshot))	\$400 (DR Aurora snapshot)
Warm Standby	No (secondary scaled down but operational)	Medium (operational, regular sync)	Strongly consistent (replication lag)	Search services (product catalog, search) (\$1,200 (reduced-scale))	\$1,200 (reduced-scale)

20-14

ElastiCache Redis Multi-AZ and Cluster Mode

ShopFlow ElastiCache Redis cluster: 3 shards, 1 primary + 1 replica per shard = 6 nodes total. If a primary fails (node-level failure), the replica in a different AZ is automatically promoted in ~30-60 seconds. If an entire AZ fails (AZ-level failure), all primaries in that AZ failover to replicas in surviving AZs. Cluster mode distributes keys across shards — horizontal scaling + HA.

ElastiCache Redis Cluster Mode CDK

```
const redisCluster = new elasticache.CfnReplicationGroup(this,"ShopFlowRedis",{
  replicationGroupDescription: "ShopFlow Redis Cluster",
  automaticFailoverEnabled: true, // promote replica if primary fails
  multiAzEnabled: true, // primaries and replicas in different AZs
  numNodeGroups: 3, // 3 shards
  replicasPerNodeGroup: 1, // 1 replica per shard (6 nodes total)
  cacheNodeType: "cache.r6g.large",
  engine: "redis",
  engineVersion: "7.0",
  atRestEncryptionEnabled: true,
  transitEncryptionEnabled: true,
  snapshotRetentionLimit: 1, // 1-day snapshot (cache is reconstructable)
  snapshotWindow: "03:00-04:00",
  preferredMaintenanceWindow: "sun:05:00-sun:06:00",
  cacheSubnetGroupName: redisSubnetGroup.cacheSubnetGroupName,
  securityGroupIds: [redisSg.securityGroupId],
});
```

■ ElastiCache Redis cluster mode: keys are distributed across shards by hash slot (16,384 total slots; 3 shards = ~5,461 slots each). This means a single key always goes to the same shard — no cross-shard queries. For ShopFlow session cache: user_session:{userId} always maps to the same shard by hash slot. Cache miss rate during a shard failover (30-60 seconds): ~33% (one shard failing = 1/3 of keys temporarily unavailable during promotion). Application must handle cache miss gracefully by falling back to Aurora for the affected keys.

20-15

S3 Cross-Region Replication for DR

ShopFlow S3 Cross-Region Replication (CRR) copies objects from the primary data lake bucket (us-east-1) to the DR bucket (us-west-2) automatically and asynchronously. Replication lag: typically <15 minutes. CRR replicates: new objects, object tags, ACLs (optional), delete markers (optional). ShopFlow replicates Bronze layer only — Silver and Gold are regenerated from Bronze via Glue ETL.

S3 Cross-Region Replication CDK

```
const primaryBucket = new s3.Bucket(this,"DataLakePrimary",{
  bucketName: "shopflow-datalake-us-east-1",
  versioning: true, // required for CRR
  replicationRules: [{
    id: "replicate-bronze-to-dr",
    destination: {
      bucket: drBucket,
      storageClass: s3.StorageClass.STANDARD_IA, // cheaper in DR (rarely accessed)
      replicationTime: { // S3 Replication Time Control (RTC): 99.99% in 15 min
        enabled: true,
        time: Duration.minutes(15),
      },
    },
    filter: {prefix: "raw/"}, // replicate Bronze layer only
  }],
  encryption: s3.BucketEncryption.KMS,
  encryptionKey: bucketKey,
});
```

S3 CRR Option	ShopFlow Choice	Reason
Replicate all objects vs prefix filter	Prefix filter: raw/ (Bronze only)	Silver and Gold are derived from Bronze; re-run Glue ETL in DR to regenerate

S3 CRR Option	ShopFlow Choice	Reason
Replicate delete markers	No	Accidental deletes should not propagate to DR; DR bucket serves as protection
S3 Replication Time Control (RTC)	Yes (15-minute SLA)	RTC guarantees 99.99% of objects replicated within 15 minutes; important for DR
DR storage class	Standard-IA	DR objects rarely accessed (only during actual disaster); Standard-IA saves costs

20-16

ShopFlow Black Friday HA Checklist

Black Friday is the highest-stakes HA event for ShopFlow. 48 hours before Black Friday: complete pre-scaling checklist. Game plan: capacity is pre-provisioned so auto-scaling is not needed during the event; all changes are frozen 72 hours before Black Friday; on-call engineer is available throughout the event.

Checklist Item	Owner	T-Before BF	Validation
Aurora: pre-provision 2 extra read replicas	DBaaS Platform	T-48h	aws rds describe-db-clusters — confirm 4 readers (2 normal + 2 extra)
Kinesis: scale OrderEvents from 8 to 50 shards (3 steps)	Platform	T-48h	aws kinesis describe-stream — confirm 50 shards ACTIVE
ECS: increase minimum tasks from 6 to 20 per service	Platform	T-24h	ECS console: confirm desired count = 20 per service
ElastiCache: add 3 extra Redis nodes	Platform	T-24h	ElastiCache console: confirm 9 nodes (6 normal + 3 extra)
Route 53: verify health checks passing	Network	T-24h	Route 53 console: all health checks GREEN
ALB: verify connection draining time set to 60s	Network	T-24h	ALB target group: deregistration delay = 60s
WAF: enable rate limiting rules (10K req/s, 5min)	Security	T-24h	WAF console: rate limit rules ENABLED with CloudWatch metric
PagerDuty: all engineers on-call rotated in	Engineering Manager	T-24h	PagerDuty: verify schedule coverage for 72-hour window
AWS Support: upgrade to Enterprise (TA) for 1 week	Engineering Manager	T-1 week	AWS console: Business Support case opened for BF readiness review

20-17

Failback — Returning to Primary Region

After DR failover, ShopFlow must fail back to us-east-1 when the region recovers. Failback is more complex than failover: data written to us-west-2 (now primary) must be replicated back to us-east-1 before traffic switches. Aurora Global Database supports reverse replication — promote us-east-1 back to primary after replication catches up. Total failback time: 30-60 minutes.

ShopFlow Failback Procedure (us-east-1 Recovery)



Failback Risk	Mitigation	ShopFlow Implementation
Data loss during reverse replication	Wait for replication lag <1 second before switching; Monitor Aurora Global Database replication lag metric; alert if >1 second	Use Aurora Global Database replication lag metric; alert if >1 second
Application cache stale state (Elasticache)	Flush Redis cache after failback; accept 2-3 minutes of stale data	Flush Redis cache after Aurora failback completion
DNS propagation delays	Low TTL (60s) ensures fast propagation; users see new endpoint within 60 seconds	Route 53 with 60 second ALB aliases permanently

20-18

SQS and SNS HA — Message Queue Resilience

SQS and SNS are managed services with built-in Multi-AZ redundancy — AWS stores SQS messages across multiple AZs automatically. ShopFlow order-processing queue uses FIFO SQS for ordered processing. Key HA considerations: dead-letter queue (DLQ) for failed messages, visibility timeout sizing, and message deduplication for FIFO queues during retries.

SQS HA Feature	How it Works	ShopFlow Configuration
Multi-AZ message storage	AWS stores messages across 3 AZs within a region automatically (for standard SQS); single AZ failure does not affect availability	Use standard SQS (not FIFO) for Multi-AZ storage
Dead Letter Queue (DLQ)	After maxReceiveCount retries, failed messages moved to DLQ for processing	OrderProcessingDLQ; maxReceiveCount=5; CloudWatch alarms for DLQ size
Visibility Timeout	Message hidden from other consumers while being processed; failure reports message appears after timeout	30 seconds; failure reports message appears after timeout
Message Deduplication (FIFO)	FIFO queues deduplicate based on MessageDeduplicationId; prevents duplicate processing	OrderProcessingFIFO; deduplication ID; prevents duplicate processing
Cross-region SQS failover	SQS does not natively replicate cross-region; use SNS to push messages to SQS in other regions	Send SNS messages to SQS in us-east-1 and us-west-2

20-19

Lambda HA — Concurrency and Error Handling

Lambda is inherently multi-AZ: AWS runs Lambda functions across multiple AZs within a region automatically. However, Lambda has its own failure modes: function errors, timeout, concurrency limits, and cold starts. ShopFlow Lambda HA: reserved concurrency prevents noisy-neighbor throttling, DLQ on all async-invoked functions, and X-Ray tracing to identify error patterns.

Lambda HA Concern	Failure Mode	ShopFlow Solution
Concurrency limit exhaustion	Lambda function throttled (429 TooManyRequests)	Request failover to other buckets; processor=100, email-notifier=2
Async invocation failures (SNS, EventBridge)	Failed Async invocations retried twice by Lambda	Services then deployed (availability) have DLQ or EventBridge
Lambda cold start during AZ failure	New Lambda containers started in surviving AZs; 500ms SnapStart (Java 11)	500ms SnapStart (Java 11) pre-initialize JVM; cold start reduced
Timeout cascades	Lambda times out calling downstream service; retries	Circuit breaker patterns (Resilience4j in Java 21): short-circuit a

Lambda SnapStart CDK — Reduce Cold Start

```
const orderProcessorFn = new lambda.Function(this,"OrderProcessor",{
  runtime: lambda.Runtime.JAVA_21,
  handler: "com.shopflow.OrderProcessorHandler::handleRequest",
  snapStart: lambda.SnapStartConf.ON_PUBLISHED_VERSIONS, // SnapStart enabled
  currentVersionOptions: {
    provisionedConcurrentExecutions: 5, // 5 pre-warmed instances always ready
  },
});
// SnapStart: Lambda takes snapshot of initialized JVM after deploy
// New instances restored from snapshot instead of cold-initializing JVM
// Cold start: 500ms-3s → <200ms with SnapStart
```

20-20 Full ShopFlow HA Architecture Summary

Complete ShopFlow High Availability architecture: 99.99% target across all layers. Every component has redundancy: 3 AZ ECS deployment, Aurora Multi-AZ with Global Database replica, ElastiCache 3-shard cluster with replicas, DynamoDB Global Tables, S3 Cross-Region Replication, Route 53 failover routing, and quarterly DR drills. Monthly HA review meeting tracks availability metrics against SLO.

Component	HA Mechanism	AZ Failure RTO	Region Failure RTO	Availability SLO
ECS Fargate Tasks	Spread across 3 AZs; Auto Scaling replaces failed tasks	1-10s (ALB detects unhealthy tasks)	1-15 min (auto-scaling across regions)	99.99% task 40s
Aurora PostgreSQL	Multi-AZ; auto-failover to reader replica in different AZ	1-120s (DNS update + read switch)	60-120 min (Global Database promotion)	99.99%
ElastiCache Redis	3-shard cluster mode; 1 replica per shard in different AZs	1-60s (primary promotion)	1-2 hours (No DR replica)	99.95% (cache recon)
DynamoDB	Multi-AZ by default; Global Tables (us-east-1 + us-west-2)	0s (in-region)	0s (active-active Global Tables)	99.99%
SQS / SNS	Multi-AZ by default; DLQ for failed messages	0s (transparent to application)	Minutes (SNS fan-out to other regions)	99.99% on queues
S3 Data Lake	Multi-AZ; versioning enabled; CRR to us-west-2	0s (transparent)	Minutes (access DR bucket)	99.999%

20-21 Service Level Objectives — Measuring Availability

ShopFlow Service Level Objectives (SLOs) define quantitative availability targets. SLOs are measured via CloudWatch composite alarms and custom metrics. Error budget: if SLO is 99.99%, error budget is 52 minutes downtime per year. When error budget is <20% remaining, all feature work stops and reliability work takes priority. Error budget is tracked weekly.

	Target	Measurement Method	Current (last 30 days)	Error Budget Remaining
Placement Success Rate	99.95% of order attempts successful	orders_created / (orders_created + order_errors)	99.97% (3 incidents, each <2 min)	78% of monthly budget
Placement Latency p99	<500ms for 99% of requests	ALB TargetResponseTime p99	380ms (within SLO)	Not applicable (latency)
Out API Availability	99.99% uptime (52 min downtime/year)	Route 53 health check success rate	99.996% (2 min downtime in last 30 days)	88% of monthly budget
Product Search Availability	99.95% (219 min downtime/year)	ALB 5xx rate < 0.05% of requests	99.98% (no incidents)	100% of monthly budget

SLO Error Budget Alarm CDK

```
// Track order success rate; alert when error budget burns too fast
const orderErrorRate = new cloudwatch.MathExpression({
  expression: "1 - (success / (success + errors))",
  usingMetrics: {
    success: ordersCreatedMetric,
    errors: orderErrorsMetric,
  },
  period: Duration.minutes(5),
});

new cloudwatch.Alarm(this, "ErrorBudgetBurn", {
  metric: orderErrorRate,
  threshold: 0.001, // > 0.1% error rate = burning budget too fast
  evaluationPeriods: 3, // 15-minute sustained burn
  alarmDescription: "Order error rate exceeds SLO; burning error budget",
  alarmActions: [sloSnsTopic.topicArn],
});
```

■ Measure SLOs from the customer perspective, not the infrastructure perspective. Infrastructure availability (Aurora Multi-AZ uptime: 99.99%) does not equal customer-visible availability (order placement success rate). A bug in order-svc code can cause 100% order failures even when all AWS infrastructure is healthy. Use synthetic canary tests (CloudWatch Synthetics) that simulate a real customer placing an order every minute — this measures actual customer experience, not just infrastructure health.

20-22

Operational Runbooks and On-Call Procedures

ShopFlow operational runbooks define step-by-step responses to common incidents. Runbooks live in Confluence, linked from PagerDuty alerts. Key runbooks: Aurora failover, ECS deployment rollback, Kinesis lag recovery, WAF false positive, and certificate expiry. On-call rotation: 2-week primary + backup; P1 (customer impact) SLA 15 minutes acknowledgment, 1-hour resolution.

Incident Type	Severity	First Response Steps	Resolution Steps	Post-Incident
Aurora writer failure	P1 (orders failing)	1. Verify via CloudWatch RDS/WriterAvailable	2. Check if auto-failover (R1 p129)s4. Verify and RDS additional details	3. Review Aurora logs for errors
ECS service crash	P1 (service unavailable)	1. Describe-services: check desired vs running	2. Check CloudWatch ECS logs for errors	3. Rollback if needed
Kinesis Iterator Age	P2 (analytics lag)	1. Check if processed	2. Check Lambda concurrency	3. Review logs
WAF blocking legitimate traffic	P2 (customer complaints)	1. Check WAF logs (Athena query)	2. Identify rule causing false positives	3. Adjust rule or whitelist

20-23

Post-Incident Review — Blameless Culture

ShopFlow post-incident reviews (PIRs) are blameless and constructive. Goal: identify systemic failures and preventive measures, not individual blame. PIR is completed within 48 hours of P1 incidents. Every PIR produces at least two action items: one immediate fix and one architectural improvement. PIRs are stored in a shared Confluence space and reviewed in monthly reliability meetings.

PIR Template — Five-Section Format

```
# Incident: Order Processing Failure — 2024-11-02 14:32 UTC
## Timeline
14:32 — PagerDuty alert: order-svc error rate > 5%
14:35 — On-call acknowledged; identified Aurora connection pool exhausted
14:42 — Temporary fix: increased max pool size from 20 to 40
14:48 — Error rate returned to <0.1%; incident resolved
Duration: 16 minutes. Orders affected: ~640 (at 40 orders/min)
## Root Cause
New product-svc deployment introduced N+1 query (SELECT per product in cart)
High cart checkout rate caused 10x more DB queries than expected
Connection pool (max 20) exhausted; new requests failed with timeout
## Contributing Factors
1. No query count metric in staging load test
```

2. Connection pool size not reviewed in deployment checklist

Action Items

1. [IMMEDIATE] Add slow query alert: >100ms query > 10% of requests (1 week)
2. [ARCHITECTURAL] Add EXPLAIN ANALYZE to pre-commit CI check (2 weeks)
3. [PROCESS] Add DB connection count to deployment health metrics (1 week)

■ Blameless PIR is not just a policy — it is an engineering culture practice. Teams that fear blame hide incidents, delay reporting, and apply quick fixes instead of root cause solutions. At ShopFlow, the PIR explicitly states "This review is for learning, not blame. Any engineer in the same situation with the same information would likely have made the same decision." This framing makes engineers comfortable reporting incidents quickly and honestly, which reduces MTTR (Mean Time To Recover) across the organization.

20-24

Well-Architected Framework — Reliability Pillar

AWS Well-Architected Framework Reliability Pillar has five design principles: (1) Automatically recover from failure, (2) Test recovery procedures, (3) Scale horizontally, (4) Stop guessing capacity, (5) Manage change through automation. ShopFlow assessment against all five principles shows high compliance after HA improvements in this chapter. Two remaining gaps: quarterly DR drills not yet scheduled, chaos experiments not yet in CI pipeline.

Reliability Principle	ShopFlow Implementation	Compliance	Gap / Next Action
Automatically recover from failure	Route 53 failover, Aurora auto-failover, ECS Auto Scaling	High	ALB health checks, Lambda Dead Queue, S3 Object Recovery;
Test recovery procedures	Monthly chaos experiments (FIS), quarterly DR drills, CodePipeline deployment	Medium	DR drills not scheduled; testing not yet executed; G
Scale horizontally	ECS Auto Scaling (CPU 60%), Aurora read replicas, DynamoDB On-Demand	High	Scaling limits for Black Friday; autoscaling
Stop guessing capacity	CloudWatch-based right-sizing, Compute Optimizer recommendations	Medium	Cost Explorer recommendations review
Manage change through automation	CDK for all infrastructure, CodePipeline CI/CD, blue/green deployment	High	Manual steps for DR checks, CloudFormation

20-25

End-to-End ShopFlow Reliability Summary

After implementing all chapters in this tutorial, ShopFlow achieves the following reliability posture: 99.99% availability target met across Tier 1 services, RTO 15 minutes for region failure (down from hours with manual recovery), RPO 1 minute (down from 24 hours with daily snapshots only), and a mature FinOps practice reducing monthly spend by 32% while maintaining SLOs. The engineering team can now confidently handle 10x Black Friday traffic without manual intervention.

Metric	Before Tutorial	After Tutorial	Improvement
Availability SLO (order placement)	99.5% (4.4 hours downtime/month estimated)	99.99% (<5 minutes/month)	From three nines to four nines
RTO (region failure)	Hours (manual Aurora promotion + DNS update)	15 minutes (automated Route 53 + scripted Aurora failover)	4 hours to 15 minutes
RPO (data loss on region failure)	24 hours (daily snapshot only)	1 minute (Aurora Global Database + DynamoDB PITR)	24 hours to 1 minute
Black Friday readiness	Manual: 8 engineers on-call scaling resources	Automated: pre-scale runbook executed T-48h	48h manual intervention to 15 min
Monthly AWS cost	\$47,000/month (over-provisioned, no RIs, no Savings Plans)	\$32,000/month (right-sized, Savings Plans, Reserved Instances)	32% reduction, \$180,000/year
Mean Time to Recover (MTTR)	~90 minutes (manual detection + diagnosis + fix)	15 minutes (automated detection, runbooks)	90 min to 15 min (83% improvement)

■ The ShopFlow tutorial pattern applies to any e-commerce platform: start with a working multi-AZ architecture, measure everything (CloudWatch, X-Ray, Athena), right-size based on data (not estimates), commit to savings plans after right-sizing, automate DR, and test it quarterly. AWS Well-Architected Tool provides a free self-assessment: answer 58 questions across 5 pillars; receive prioritized recommendations. Run Well-Architected review annually and after major architectural changes.

20-26

Proactive Reliability — Auto Scaling Predictive Mode

Reactive Auto Scaling (scale when CPU > 60%) adds latency — it takes 60-90 seconds for new ECS tasks to start and pass health checks before receiving traffic. During that window, existing tasks are overloaded. Predictive Auto Scaling

uses ML to learn traffic patterns (daily/weekly cycles) and pre-scales capacity before traffic arrives — not after.
ShopFlow: Black Friday follows predictable traffic ramp-up; predictive scaling pre-provisions 15 minutes early.

Predictive Auto Scaling CDK (ECS via Application Auto Scaling)

```
// ECS does not natively support Predictive Auto Scaling (that is EC2 ASG only)
// For ECS: use EventBridge Scheduler for pre-scaling at known traffic peaks
// Scheduled scaling: increase ECS tasks at 8am (business hours start)
new applicationautoscaling.ScalableTarget(this, "OrderSvcScalable", {
  serviceNamespace: applicationautoscaling.ServiceNamespace.ECS,
  scalableDimension: "ecs:service:DesiredCount",
  resourceId: `service/${cluster.clusterName}/${orderService.serviceName}`,
  minCapacity: 6,
  maxCapacity: 50,
}).scaleOnSchedule("MorningScaleOut", {
  schedule: applicationautoscaling.Schedule.cron({hour:"7",minute:"45"}),
  minCapacity: 15, // pre-scale to 15 tasks at 7:45am (before 8am rush)
});

// Black Friday: pre-scale to 40 tasks starting at 11pm night before
.scaleOnSchedule("BlackFridayScaleOut",{
  schedule: applicationautoscaling.Schedule.cron({
    month: "11", day: "28", hour: "23", minute: "0"}),
  minCapacity: 40, // Black Friday pre-scale (2024-11-28 23:00 = night before BF)
});
```

Scaling Approach	Trigger	Lead Time	ShopFlow Use
Reactive (CPU-based)	CPU > 60% for 3 minutes	3 min (alarm) + 1 min (ECS task start + health check)	Normal day-to-day variance; hardware failure
Scheduled (EventBridge)	Calendar time (7:45am, Black Friday date)	Pre-scales 15 min before traffic; no lag during traffic ramp	Daily morning ramp; Black Friday
Manual (CDK redeploy)	Engineer runs CDK deploy with higher minCapacity	5-10 min (CDK deploy); requires engineer action	One-off events (flash sales, marketing)

20-BP

Best Practices and Anti-Patterns

- ✓ Define RTO and RPO explicitly per service tier — not all services need the same DR level; Tier 1 (orders, payments) warrants Aurora Global DB; Tier 4 (admin tools) can be recreated from CDK in hours
- ✓ Use Aurora Global Database for relational data that requires sub-second RPO — ~1 second replication lag + <1 minute promotion; far better than snapshot restore (hours)
- ✓ Enable DynamoDB PITR on all production tables — \$0.20/GB-month for continuous backup; allows restore to any second in the last 35 days; prevents catastrophic data loss from accidental delete or application bug
- ✓ Multi-AZ ECS placement with spreadTasksByAz: true — distributes tasks across 3 AZs; surviving 2 AZs absorb 100% of traffic when one AZ fails (design for n-1 AZ capacity)
- ✓ Set Route 53 TTL to 60 seconds for ALB aliases — lower TTL enables faster failover DNS propagation; 300-3600 second TTL causes long failover delays even with sub-minute health checks
- ✓ Test DR quarterly with a real production drill — DR runbooks that are never tested will fail during actual disasters; GameDays reveal gaps before they matter
- ✓ Use AWS Backup Vault Lock for production backups — prevents accidental or malicious deletion of backups; critical for ransomware protection; set to compliance mode after testing
- ✓ Pre-warm DR region ECS tasks during normal operation — keep 1-2 tasks per service running in DR region; eliminates cold start during failover (Fargate cold start: 30-90 seconds per task)
- Never test DR in production without a fallback plan — always have a rollback procedure before starting any chaos experiment; document expected behavior vs actual behavior
- Do not rely on Aurora snapshot restore as primary DR — snapshot restore creates a new cluster (RTO hours, not minutes); use PITR or Global Database for production DR
- Avoid single-AZ data stores for any Tier 1 or Tier 2 service — a single-AZ ElastiCache or RDS without Multi-AZ is a single point of failure; the cost savings do not justify the risk for customer-facing services

■ Do not confuse HA (stay available during failure) with DR (recover after catastrophic failure) — both are needed; HA handles AZ failures automatically; DR handles region failures with human action and data restoration

20-QR

Quick Reference and Interview Q&A

Concept	Key Value / Rule
RTO vs RPO	RTO (Recovery Time Objective): how long the system can be DOWN before recovery. RPO (Recovery Point Objective): how much data loss is acceptable.
Aurora Global Database failover	Managed failover (planned): <1 minute; write traffic rerouted to secondary region. Unplanned failover (region outage): ~15 minutes; data restored from primary region.
Multi-AZ vs Multi-Region	Multi-AZ: within one AWS region, data replicated across 2-3 Availability Zones; handles AZ failure (hardware, power, network). Multi-Region: data replicated across multiple AWS regions; handles region failure.
DynamoDB Global Tables conflict resolution	Write conflict resolution: if the same item is written in us-east-1 and us-west-2 within the replication window (~1 second), the latest write wins.
Chaos Engineering best practices	1. Start with low-blast-radius experiments (non-prod, single service). 2. Define hypothesis first: "If we terminate 5% of EC2 instances, the system will remain available." 3. Automate experiments.
AWS Backup Vault Lock	Vault Lock prevents: deletion of backups before retention period expires, modification of backup policies, and disabling of the vault.